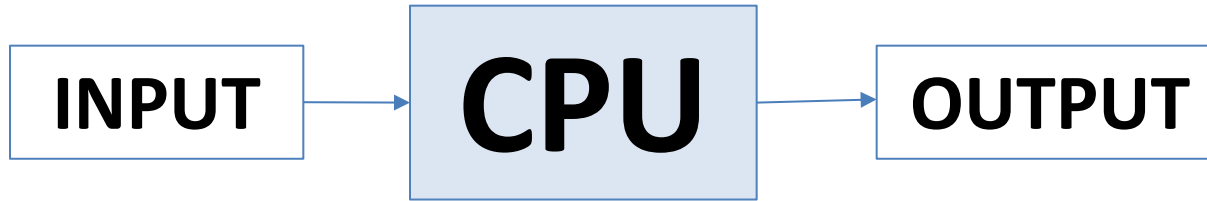


CPSC 375 High-Performance Computing

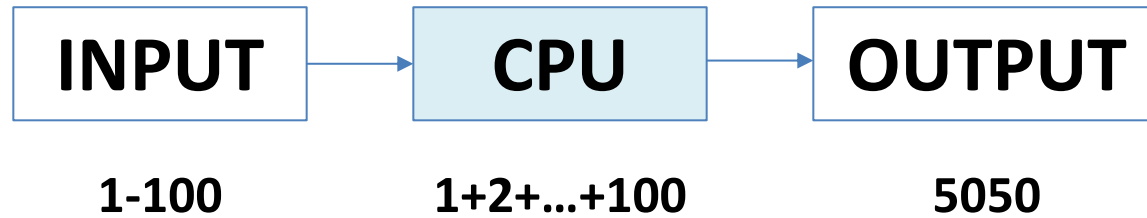
Lecture I

Course Overview

A Traditional Computer



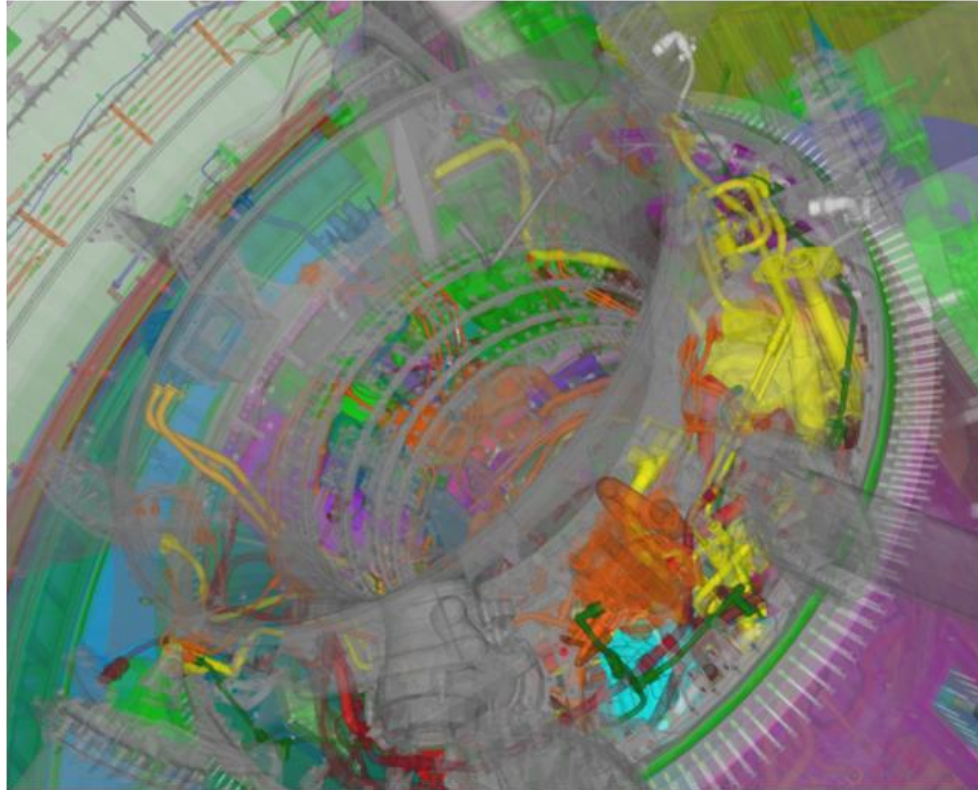
Serial Computing



Why Computing Matters To Science?

- Traditional scientific and engineering paradigm
 - Do theory
 - Design
 - Build system
 - Perform experiments
 - Analyze data
- Real phenomena are:
 - Too hard, e.g., N-Body Problem
 - Too expensive, e.g., crash test of passenger jets
 - Too slow, e.g., climate model
- Computing can help replace some of key steps.

Boeing 777

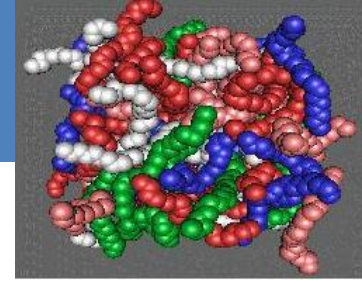


Some Challenging Problems

- Drug design
- Mapping human genome
- Weather forecast
- Understanding biological structures
- Understanding new materials
- Modeling astrophysical phenomena
- Understanding vehicle dynamics
- Modeling earthquake
- Web mining
- Database transaction processing
- ...

So, how demanding are these problems anyway?

Molecular Dynamics



- 1,000,000 particles interacting every microsecond
- 3 coordinates, 3 velocities, 3 forces
- Total calculation to simulate 1 second $> 10^{18}$ operations.
- With a typical computer running at 3.0 GHz, it would take about 10 years to complete!

Global Climate Modeling



- Earth's surface approx. $5 \times 10^8 \text{ km}^2$
- Considering 1 location point per km^2 with 15 levels (ground level, up to 14 km)
- Six data values – temperature, humidity, wind, latitude, longitude, height
- Updating once every minute would require 3 billion operations per second.
- Forecasting weather over 7 days using a typical computer would take 100 days! For a 2T computer, it would take about 10 minutes.

Need For Speed

Solve problems much *faster*!

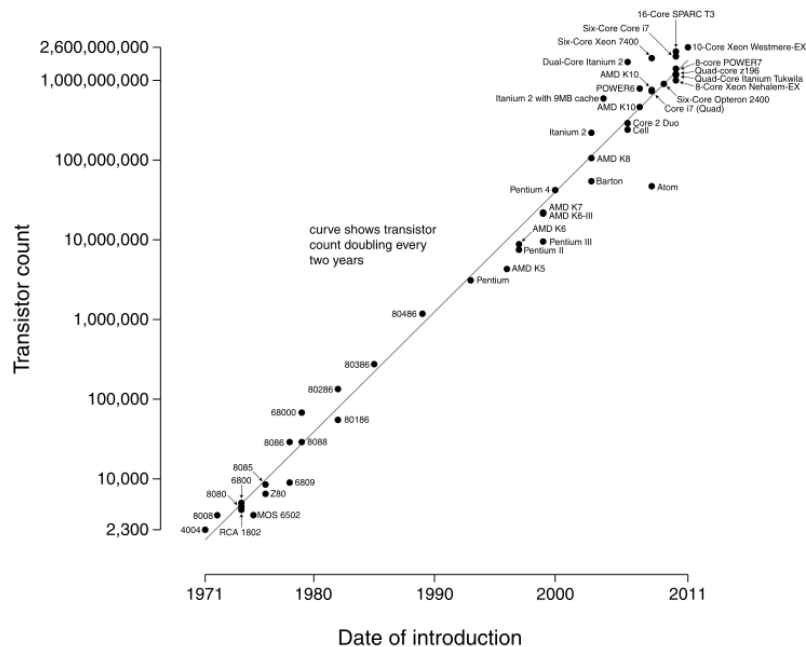
Solve much *bigger* problems!

Why can't we make faster computers?

Moore's Law

“The number of transistors on a processor chip will roughly double in every two years.”

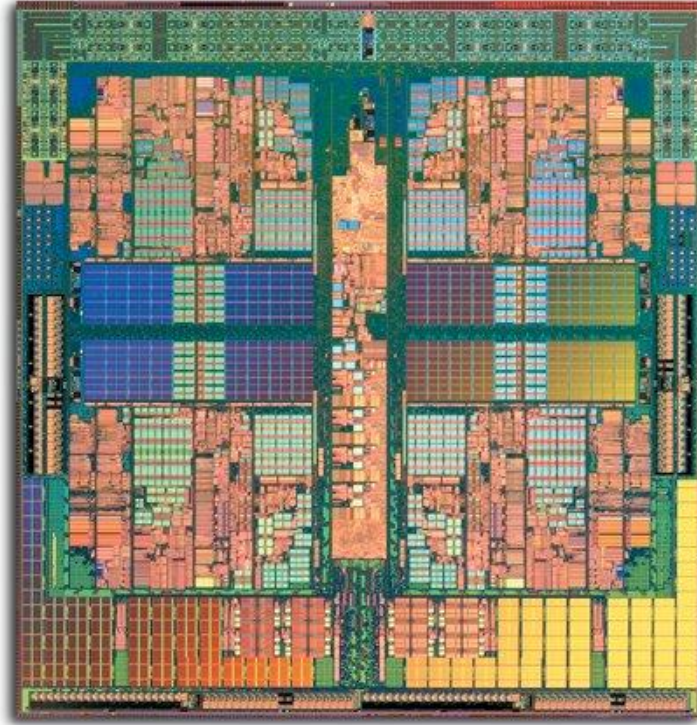
Microprocessor Transistor Counts 1971-2011 & Moore's Law



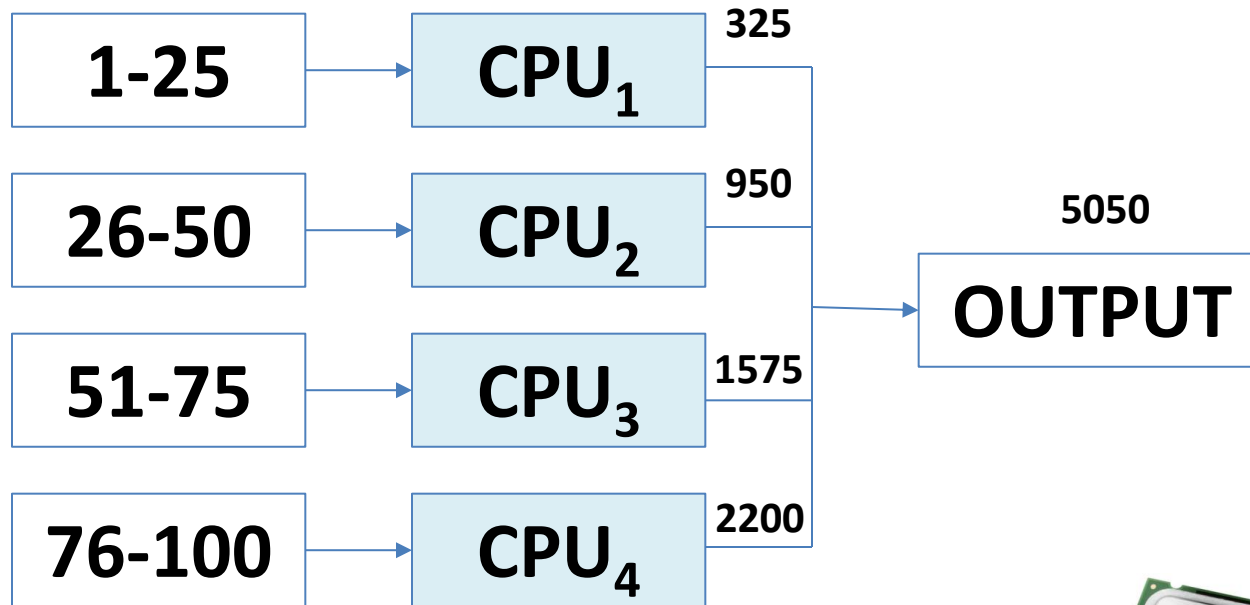
Hitting the Wall

- Limitations in device physics
- Key Issues
 - Higher clock speed (frequency)
 - Power consumption
 - Heat dissipation
 - Memory gap
- Solution: Multicore processors

A Typical Quadcore Processor



Parallel Computing



High-Performance Computing

- Important problems require powerful computers ...
 - Powerful computers must be parallel.
 - Increasing importance of educating parallel programmers
 - Some parallel programmers need to be performance experts
 - Architectures in this class
 - Multi-cores, SMPs, networked clusters, GPUs (possibly S'27)
- Developing high-performance parallel applications

High-Performance Computing, cont'd

- Solving complex problems computationally using
 - High-performance computers
 - High-performance connections
 - High-performance I/O

aka

- Supercomputing
- Parallel computing
- Cluster computing
- Grid computing
- GPU computing
- etc.

The Fastest Supercomputer (2026)



- **El Capitan** by Hewlett Packard Enterprise
- @Lawrence Livermore National Laboratory
- 1.8 exaflops [1 exaflop: quintillion (10^{18}) floating-point operations per second]
- Supports the National Nuclear Security Administration
- 11,340,000 cores, 30 megawatt

Course Objectives

- Learn how to program parallel processors and systems
 - Learn how to think in parallel and write correct parallel programs
 - Achieve performance and scalability through understanding of architecture and software mapping
- Significant hands-on programming experience
 - Develop real applications on real hardware
 - Develop parallel algorithms
- Discuss the current parallel computing context
 - Contemporary programming models and architectures
 - Future directions

Why Does This Course Matter?

- Multi-core and many-core era is here to stay
 - Why? Technology trends
- Many programmers will be developing parallel software
 - But still not everyone is trained in parallel programming
 - Learn how to put all these vast machine resources to the best use!
- Our focus
 - Teach core concepts
 - Use common programming models
 - Discuss broader spectrum of parallel computing

Instructor

Prof. Peter Yoon

- Office: MECC 127
- Phone: 860-297-2461
- Email: peter.yoon@trincoll.edu
- Office: MWF 2:00 – 3:00 p.m. (by appointment)

Requirements

- Assignments
 - Weekly
 - No late work will be accepted
- Exams
 - Exam I: February 20
 - Exam II: March 27
 - Final Exam: 3 p.m., May 7

Grading

Assignments	30%
Exams (2)	30%
Final Exam	30%
Attendance	10%

Course Policy

- Attendance very important!
- Learning disability
- Classroom rules
 - Laptops: permitted
 - Electronic communications: **forbidden!**
 - No Web surfing, email, texting, cell phone calls, etc.
- See course website for details.

Academic Honesty Guidelines

- Strictly prohibited
 - Giving and receiving help in the actual development of code or writing of an assignment
 - Looking at another person's code
 - Showing your code to another person
 - Sharing a copy of all or part of your code, regardless of whether that copy is on paper or in a computer file
 - Turning in the work of any other person(s) (former students, friends, people on the Internet, etc.) or code generated by a machine and representing it as your own work
 - Fabricating compilation or execution results
- Penalty for cheating
 - A failing grade (F) for the course
 - Will be asked to appear at an academic dishonesty hearing.

